

CLASS 07 / OCT 6 / U04

ARRAYS 2

IF · SWITCH · PATTERN

A RULE THAT READS THE INDEX

NOTHING DUE TODAY

1 · 2.1 · 2.2 · 3.1 · 3.2 DUE ON TEAMS SUN OCT 18, 11:59 PM

LAST WEEK: EVERY COPY THE SAME

Line, grid, cube, curve — the loop placed one module at every position.

THE COUNTERS YOU ALREADY HAVE

i → along the row

j → which row

k → which layer

$$\text{index} = \text{i} + \text{j} * \text{nx} + \text{k} * \text{nx} * \text{ny}$$

TODAY

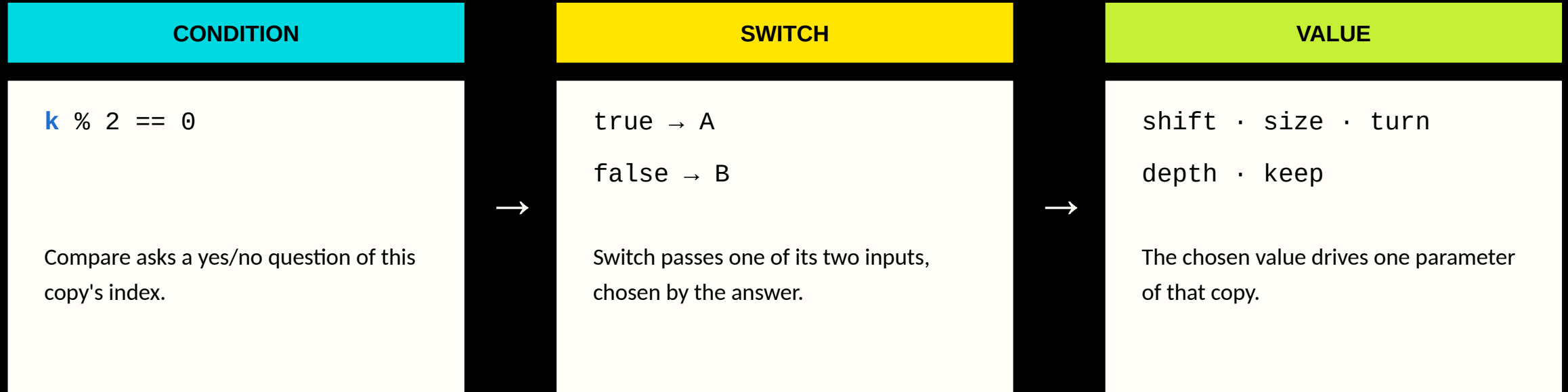
A rule reads those counters and decides something about each copy.

Same loop, same positions. The only new thing is a question asked of every copy — and an answer that changes one of its parameters.

THE INDEX WAS A COUNTER. TODAY IT BECOMES AN INPUT.

A CONDITION, A SWITCH, A VALUE

Three nodes. In the class file: Compare → Switch → the input it drives.



```
value = A if condition else B      # the whole idea, in one line
```

A CONDITION IS TRUE OR FALSE. A SWITCH TURNS THAT INTO ONE OF TWO VALUES.

MODULO — THE TEST THAT REPEATS

% gives the remainder. It turns a counter that keeps growing into one that cycles.

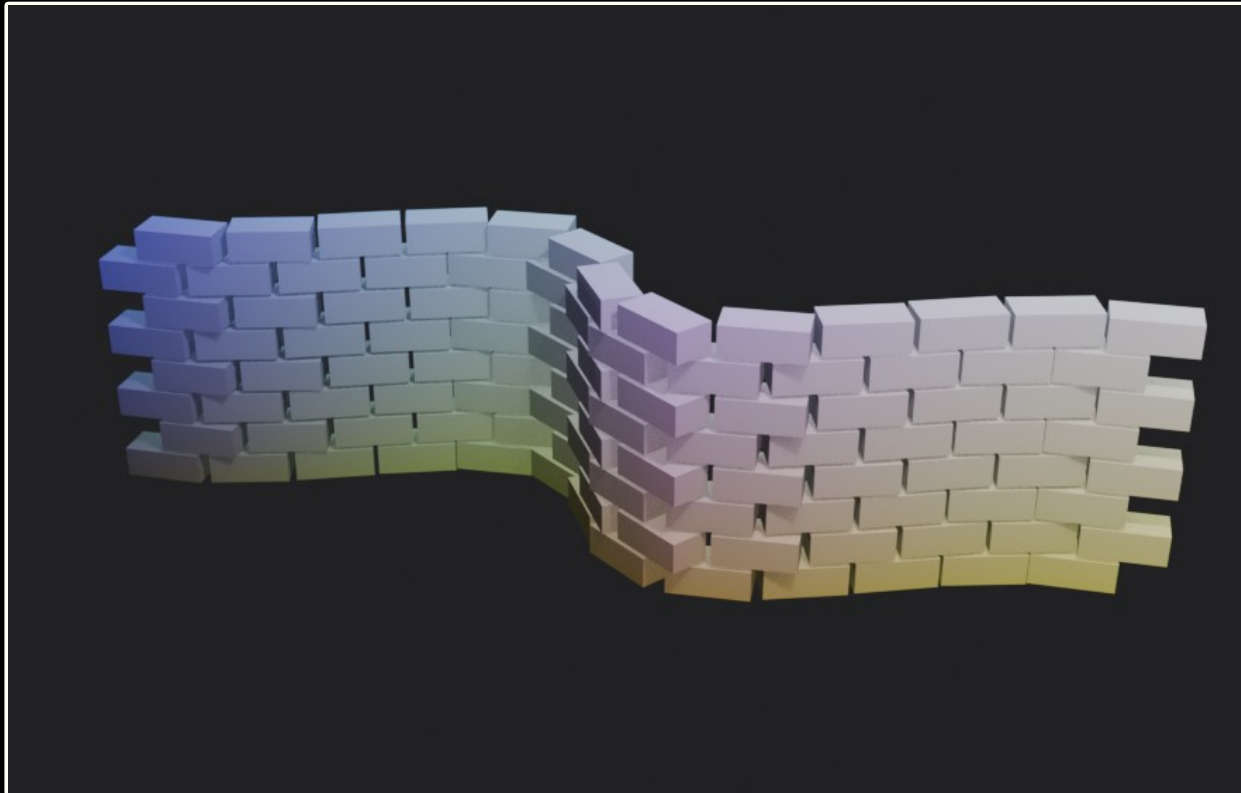
i	0	1	2	3	4	5	6	7	8
i % 2	0	1	0	1	0	1	0	1	0
i % 3	0	1	2	0	1	2	0	1	2
i % 3 == 0	T	F	F	T	F	F	T	F	F

Pink is where the test is true: every odd copy, then every third copy. Read the row before you build the rule.

ODD/EVEN IS % 2. EVERY THIRD IS % 3 == 0. EVERY PATTERN TODAY STARTS HERE.

01 · BRICK WALL — EVERY OTHER COURSE SHIFTS

Arrays Part 2 · Brick wall · the row from last week, stacked in courses along a curve.



ONE RULE ON k

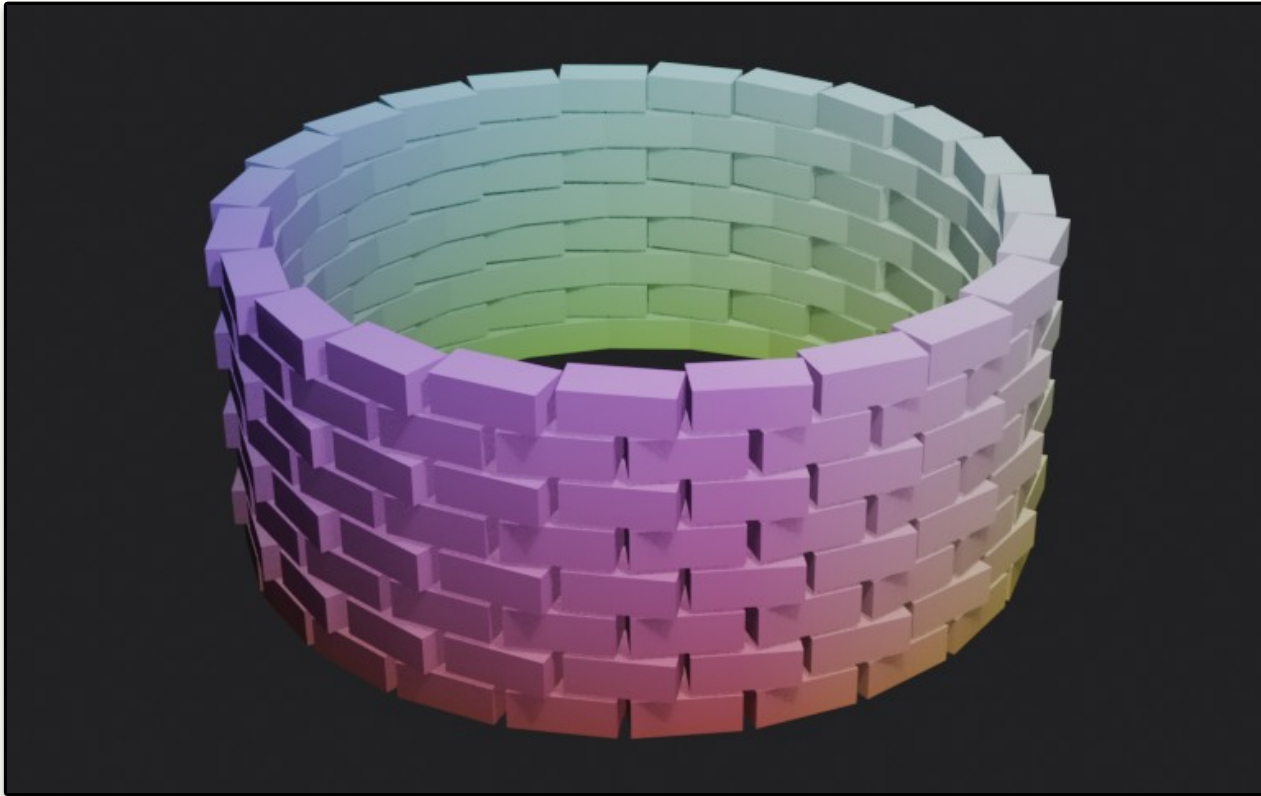
```
for  $k$  in range(courses):  
    shift = ( $k$  % 2) * brick / 2  
    for  $i$  in range(count):  
        p = point_on(curve, t)  
        p = p + d * shift  
        place(brick, p, z= $k$ *h)
```

k is the course. Odd courses slide half a brick, even ones don't. That single rule is what makes it a bond.

THE FIRST PATTERN: ONE CONDITION ON THE LAYER INDEX, ONE VALUE IT CHANGES.

SWAP THE CURVE FOR A CIRCLE

Same scene, same rule. Only the path that supplies the positions changed.



TWO JOBS, KEPT APART

The curve answers: where is copy i?

The rule answers: what is different about this copy?

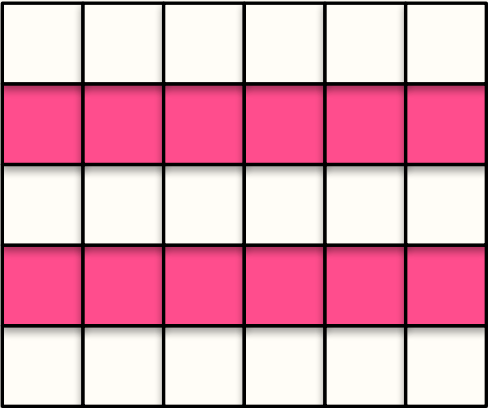
Because they are separate, you can change either one without touching the other. That is what makes a rule reusable.

POSITION FROM THE LOOP. DIFFERENCE FROM THE RULE.

SHEET 2 · FOUR FAÇADE PATTERNS

Your 3.1 module in a 2D array. Pink = the rule is true for that copy.

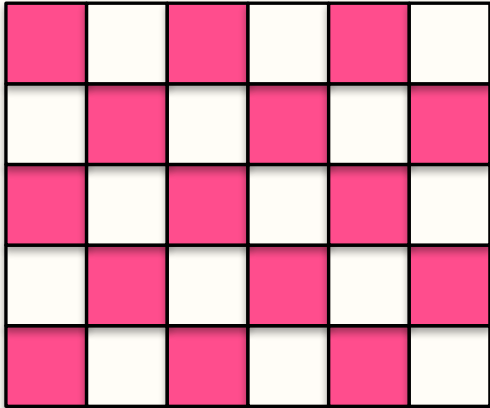
ALTERNATE ROWS



$j \% 2 == 1$

true → turn 90°

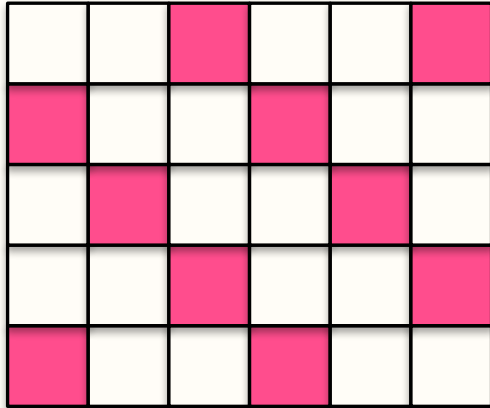
CHECKERBOARD



$(i + j) \% 2 == 0$

true → scale up

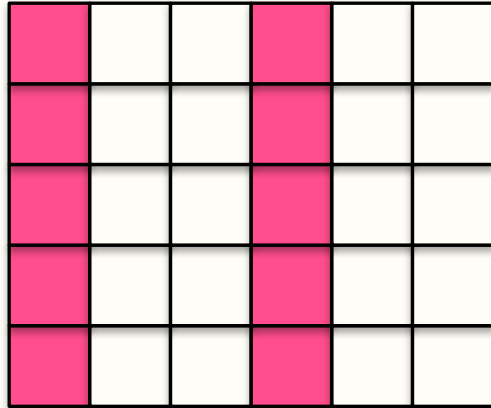
DIAGONAL BANDS



$(i + j) \% 3 == 0$

true → push out

EVERY THIRD COLUMN



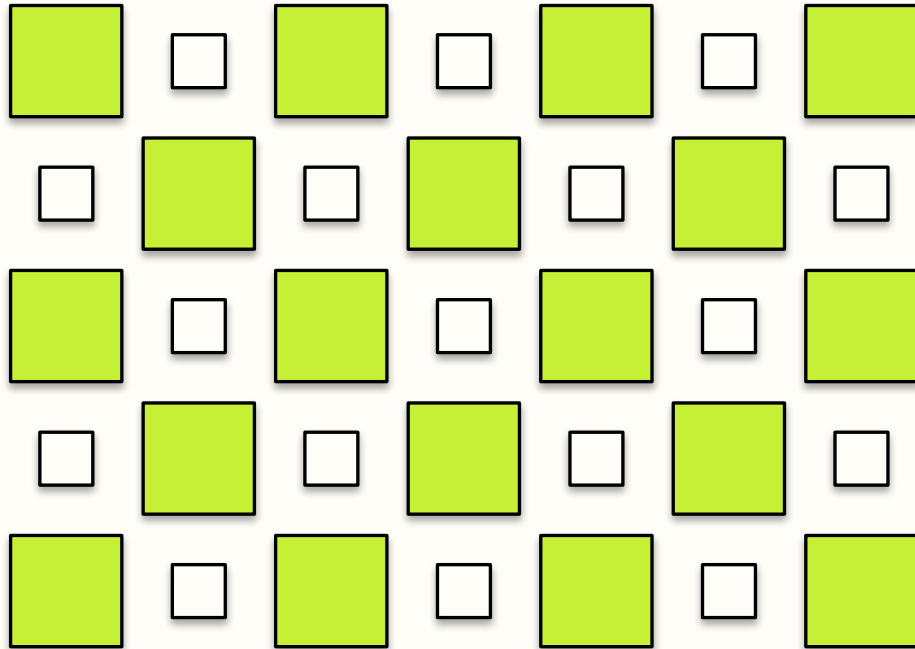
$i \% 3 == 0$

true → remove

CAPTION EACH ONE: THE RULE, AND WHAT IT DOES WHEN IT IS TRUE AND WHEN IT IS FALSE.

A RULE PICKS ONE OF TWO VALUES

Keep or remove is only one case. Any parameter with two settings can be switched.



THE CHECKERBOARD, DRIVING SIZE

```
checker = (i + j) % 2 == 0
```

```
size = big if checker else small
```

Two values: large and small. The rule decides which one each copy gets.

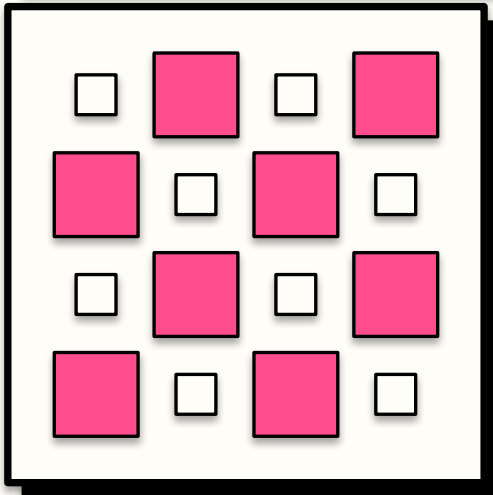
Swap size for rotation (0° or 45°), depth, an aperture — or 'there' and 'not there'. Same rule, different parameter.

TRUE → ONE VALUE. FALSE → THE OTHER. ONE RULE DRIVES ONE PARAMETER.

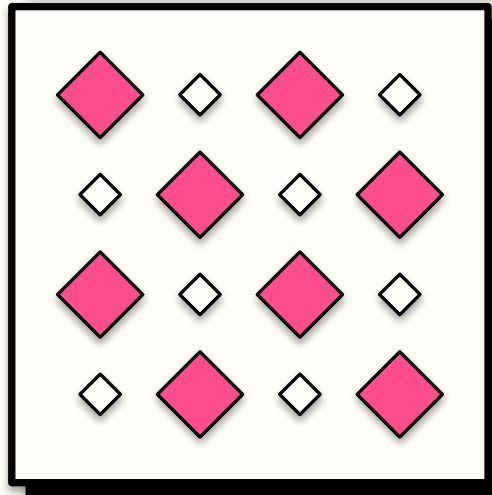
SHEET 3 · THE SAME RULES IN 3D

A 3D array of cubes, read one layer at a time. Two rules: size from $i + j + k$, turn from k .

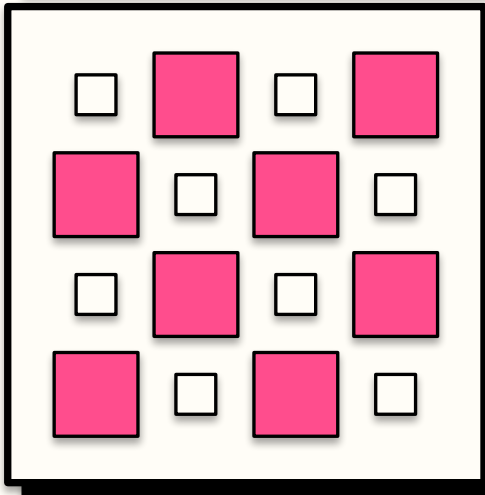
LAYER $k = 0$



LAYER $k = 1$



LAYER $k = 2$



READ THE STACK

k in the size rule: the checkerboard flips every layer.

k in the turn rule: odd layers turn 45° .

Leave k out and every layer is identical.

```
size = big if ( $i + j + k$ ) % 2 == 0 else small
```

```
turn = 45 if  $k$  % 2 else 0
```

PUT k IN THE RULE AND THE PATTERN CHANGES LAYER BY LAYER, NOT JUST ACROSS.

SHEET 3, WRITTEN OUT

The cube loop from last week, with the rules added inside it.

```
for k in range(layers):
    for j in range(rows):
        for i in range(cols):
            checker = (i + j + k) % 2 == 0
            size = big if checker else small
            turn = 45 if k % 2 else 0
            place(cube, i*dx, j*dy, k*dz,
                  size=size, turn=turn)
```

WHAT SHEET 3 SHOWS

Two patterns, each in isometric, from the same view.

At least one rule reads k.

At least one switches a value — not only keep or remove.

Caption: the rule, the parameter, its value when true and when false.

A HEIGHT FUNCTION OF X AND Y IS OPTIONAL NOW. THE RULES ARE NOT.

REQUIREMENTS — 3.2 · 1/2

Computational understanding 30% · Visual communication 30% · Technical execution 25% · Exploration 15%.

01

Submit exactly three 17 x 11 inch landscape sheets together as one assignment.

02

Sheet 1 — four diagrams: your 3.1 module alone, then in a 1D, a 2D and a 3D array. Use the same module throughout, and label the counts, spacing and axes.

03

Sheet 2 — four façade-pattern diagrams: apply if/Switch rules to a 2D array of your module to make four different patterns. Each diagram names its rule and what it changes.

04

Sheet 3 — 3D conditional patterns: make a 3D array of cubes and use if/Switch rules on its row, column and layer index to drive a parameter — the rule chooses one of two values, such as large or small, 0° or 45°, low or raised: one value when true, the other when false. Make two different patterns, each shown in isometric from the same view. At least one rule must use the layer index, and at least one must switch a parameter value rather than only keep or remove cubes. A height function of X and Y is optional.

05

Reuse the 3.1 panel on Sheets 1–2. On Sheet 2 you may change or remove selected copies to make a pattern. Use cubes, not the panel, on Sheet 3.

REQUIREMENTS — 3.2 · 2/2

Computational understanding 30% · Visual communication 30% · Technical execution 25% · Exploration 15%.

06

Keep the diagrams simple, with the same view and material wherever studies are compared. Keep the lighting consistent; the four façade patterns don't need separate polished renders. Choose a suitable view for each array, and use isometric for the 3D patterns.

07

Build the repetition from basic nodes, starting with a count small enough to check by eye; there is no minimum number of copies.

08

Make every array fully parametric: expose row, column and layer counts and spacing in each used direction. Changing those inputs must rebuild the array and resize the panel or cube to fit its new cell without manual remodeling or repositioning. Preserve the intended gaps or edge contact.

09

Use materials from the library the instructors provide, or make your own. Note which materials you used and any changes; pack any textures. Materials must keep the spacing, depth and repetition of the array readable. The library will be posted before this assignment is due; making your own is equally fine.

OPTIONAL PRACTICE

In the class file, not on the sheets. Each one is a rule you already know.

HEXAGONAL

Shift every other row by half a cell —
the brick wall's $j \% 2$, in plan.

RADIAL

Position from radius, count and angle
step. The index picks the angle.

SINE VOLUME

Height from a function of X and Y.
Amplitude, frequency and phase,
written down.

DO THESE AFTER SHEETS 2 AND 3 WORK — NOT INSTEAD OF THEM.

NOW — SHEETS 2 AND 3

Start at a count you can check by eye.

✓

Brick wall: $k \% 2$ shifting every other course, on the curve, then on a circle

✓

Your module in a 2D array with one if/Switch rule working on it

✓

Four façade rules for Sheet 2, each captioned with what it changes

✓

A 3D array of cubes where a rule switches size, rotation or height

✓

At least one rule that reads the layer index k

A RULE YOU CAN SAY OUT LOUD IS A RULE YOU CAN CAPTION.

BEFORE NEXT CLASS

NEXT

Oct 13: supported practice, nothing due. Bring all three 3.2 sheets and anything still open.

DUE

Everything through Arrays — 1, 2.1, 2.2 (a photo of your print), 3.1 and 3.2 — on Teams by Sun Oct 18, 11:59 PM. Your midterm grade is the accumulated grade of all five.

WATCH

Rhino + Grasshopper: GH list, Arrays part 1 and part 2 are on the lesson page. The Blender Arrays part 2 video is coming.

THEN

Oct 20: façade attractors — 3.3 starts.